

AIQT CLI

Milestone 5 Build Specification v0.2

Checkpoint Engine

Build-planning document for implementing AIQT Milestone 5. Not a runtime document model for AIQT user projects.

Field	Value
Product	AIQT CLI
Specification	Milestone 5 Build Specification
Version	v0.2
Milestone	M5 - Checkpoint Engine
Aligns With	AIQT MVP Specification v0.6 and Technical Architecture Specification v0.3
Depends On	M1 - Canonical State Foundation, tag m1-canonical-state-foundation M2 - Project Context Capture, tag m2-project-context-capture M3 - Work Graph Planning Foundation, tag m3-work-graph-planning M4 - Agent Handoff Packet Engine, merge commit c5b42ff, tag m4-agent-handoff-packet
Primary Agent Target	Claude Code or equivalent coding agent
Implementation Status	Ready for final review before build handoff
Document status	v0.2 review candidate. Resolves deterministic next-command precedence, starting point, literal tag names, current pointer rules, CheckpointIssueSchema distinction, and checkpoint retry guidance.

Document status: Ready for final review before build handoff. v0.2 resolves the M5 v0.1 review findings and keeps scope limited to checkpoint capture.

Contents

- 1. Source Alignment
- 2. Milestone Objective
- 3. Checkpoint Model Decision
- 4. Commands in Scope
- 5. Out of Scope
- 6. Behavioral Position in the Workflow
- 7. Checkpoint Preconditions
- 8. Checkpoint Input Modes
- 9. Checkpoint Input Contract
- 10. Completion Decision Rules
- 11. Dependency Readiness Recalculation
- 12. State Transition Rules
- 13. Canonical Mutation Rules
- 14. Runlog Rules
- 15. Command Behavior
- 16. Data and Schema Contracts
- 17. Error Handling and Exit Codes
- 18. Repository Implementation Scope
- 19. Runtime and Package Impact
- 20. Testing Scope
- 21. Implementation Tasks
- 22. Definition of Done
- 23. Agent Execution Packet
- Appendix A - Minimum Test Matrix
- Appendix B - Example Checkpoint Input File
- Appendix C - Example JSON Output

1. Source Alignment

This document is derived from AIQT MVP Specification v0.6, AIQT Technical Architecture Specification v0.3, and the completed milestone sequence M1 through M4. It assumes M4 is merged to main at commit c5b42ff and tagged as m4-agent-handoff-packet. The repository is expected to contain a full aiqt next packet engine that sets `currentWorkUnitId`, stores `lastAgentPacket` metadata, marks one work unit in `_progress`, and recommends aiqt checkpoint. Milestone tags are literal repository tags, not title-derived names. Use these actual tags: `m1-canonical-state-foundation`, `m2-project-context-capture`, `m3-work-graph-planning`, and `m4-agent-handoff-packet`.

The governing design remains:

```
state -> graph -> packet -> checkpoint
not:
spec -> milestone file -> task file -> review file
```

M5 implements the checkpoint step. It must not implement review, export, replanning, autonomous execution, or source-code modification. It captures the result of one external agent execution cycle and advances workflow state deterministically.

1.1 v0.2 Change Summary

- Defines deterministic `nextRecommendedCommand` precedence: `needs_review` takes priority over continuing to another ready work unit.
- Adds Starting Point details to the agent execution packet.
- Clarifies that milestone tags are literal repository tags and must not be inferred from milestone titles.
- Makes `currentMilestoneId` behavior deterministic after successful checkpoints.
- Clarifies that `CheckpointIssueSchema` and `CommandResult Issue` are intentionally different types.
- Changes completion-gate failure guidance to retry aiqt checkpoint rather than jump to review.

2. Milestone Objective

Milestone 5 implements aiqt checkpoint as the checkpoint capture engine. It turns the result of the agent execution cycle started by aiqt next into a durable checkpoint record, updates the active work unit status, recalculates dependency readiness, and recommends the next workflow action.

Milestone 5 must prove that AIQT can:

- run aiqt checkpoint against an initialized project with a current in `_progress` work unit;
- ingest a structured checkpoint result file;
- validate checkpoint input deterministically;
- create a checkpoint record in `state.json`;
- transition the selected work unit from in `_progress` to `done` or `needs_review`;
- recalculate planned work units that become ready after dependencies are satisfied;
- clear `currentWorkUnitId` after checkpoint capture;
- preserve `lastAgentPacket` metadata as historical context;
- append checkpoint and status-change runlog events;
- return deterministic human-readable and JSON output;
- preserve all M1, M2, M3, and M4 behavior.

3. Checkpoint Model Decision

M5 treats checkpoint capture as a state transition, not as a review engine. The user or agent provides a structured result for the current work unit. AIQT records that result and applies deterministic workflow rules. It does not inspect source code, run validation commands automatically, or judge implementation quality beyond the explicit checkpoint input.

Decision: aiqt checkpoint writes a checkpoint record and advances workflow state only when there is an active `currentWorkUnitId` that points to an in `_progress` work unit created by aiqt next.

Reason: M4 starts the execution cycle. M5 closes that cycle. This keeps AIQT resumable and prevents accidental checkpointing of a work unit that was never handed off.

4. Commands in Scope

```
aiqt checkpoint
aiqt checkpoint --json
aiqt checkpoint --from-file <path>
aiqt checkpoint --from-file <path> --json
aiqt checkpoint --example
```

File mode is the primary M5 mode. Interactive checkpoint intake and direct granular flags are intentionally deferred to avoid expanding the CLI surface before the checkpoint schema is proven.

`aiqt checkpoint --example` prints a sample checkpoint input JSON to stdout and exits with code 0. It does not require `.aiqt/` and must not mutate files. If `--json` is combined with `--example`, reject the combination with exit code 3.

5. Out of Scope

- `aiqt review` implementation;
- `aiqt export` implementation;
- automatic validation command execution;
- automatic source-code inspection;
- autonomous agent execution or process spawning;
- Claude Code, OpenAI, Codex, GitHub Copilot, or GitHub API integration;
- replanning or modifying the work graph structure;
- creating new milestones, work units, or dependencies;
- marking multiple work units complete in one command;
- checkpointing a work unit that is not `currentWorkUnitId`;
- creating markdown files by default;
- AIQT.md, AGENTS.md, CLAUDE.md, docs/, .milestones/, or .tasks/;
- exported status reports or human documentation;
- full quality review or acceptance-audit engine.

6. Behavioral Position in the Workflow

M1: init/status/next -> M2: update -> M3: plan -> M4: next packet -> M5: checkpoint -> M6: review/export

After M4, one work unit is `in_progress`, `currentWorkUnitId` points to that work unit, `lastAgentPacket` stores packet metadata, and `nextRecommendedCommand` is `aiqt checkpoint`.

After M5, the active work unit is no longer `in_progress`. It becomes `done` or `needs_review`. `currentWorkUnitId` is cleared. M5 uses deterministic next-command precedence: if the checkpointed work unit ends as `needs_review`, `nextRecommendedCommand` = `aiqt review` even when other ready work units exist; else, if the checkpointed work unit ends as `done` and at least one ready work unit exists after dependency recalculation, `nextRecommendedCommand` = `aiqt next`; else, if the checkpointed work unit ends as `done` and no ready work unit exists, `nextRecommendedCommand` = `aiqt review`.

7. Checkpoint Preconditions

Before capturing a checkpoint, `aiqt checkpoint` must validate these workflow preconditions:

Precondition	Failure behavior
<code>.aiqt/</code> exists and canonical files are readable	Exit code 3, status failed, recommend <code>aiqt init</code> or <code>repair state</code> .
<code>project.json</code> and <code>state.json</code> validate against schema and version rules	Exit code 3, status failed.
<code>state.workGraph</code> is empty and <code>planningContextReady</code> is false	Exit code 2, status blocked, recommend <code>aiqt update</code> . No mutation.
<code>state.workGraph</code> is empty and <code>planningContextReady</code> is true	Exit code 2, status blocked, recommend <code>aiqt plan</code> . No mutation.
<code>state.currentWorkUnitId</code> is non-null	Exit code 2, status blocked, recommend <code>aiqt next</code> . No mutation.
<code>currentWorkUnitId</code> references an existing work unit	Exit code 3, status failed. No mutation.

Referenced current work unit has status = in_progress	Exit code 3, status failed. No mutation.
state.lastAgentPacket exists and references currentWorkUnitId	Exit code 3, status failed. No mutation.
Checkpoint input is supplied after workflow preconditions pass	Exit code 10, status needs_input, recommend aiqt checkpoint --from-file <path>. No mutation.

8. Checkpoint Input Modes

8.1 File mode

```
aiqt checkpoint --from-file checkpoint.json
aiqt checkpoint --from-file checkpoint.json --json
```

File mode loads a structured checkpoint input JSON, validates it, derives the final status when not supplied, creates the checkpoint record, updates state, and appends runlog events.

8.2 No-input mode

```
aiqt checkpoint
```

When an active work unit exists but no --from-file argument is supplied, aiqt checkpoint returns status = needs_input, requiresHumanInput = true, exitCode = 10, and nextRecommendedCommand = aiqt checkpoint --from-file <path>.

8.3 Example mode

```
aiqt checkpoint --example
```

Example mode prints a sample checkpoint input JSON. It must not read .aiqt/, write state, append runlog events, or create export files. If --json is supplied with --example, reject the combination with exit code 3.

9. Checkpoint Input Contract

M5 accepts JSON only. YAML, markdown, natural language transcripts, and pasted full agent logs are out of scope for M5. The checkpoint input summarizes the result of one already-started work unit.

```
{
  "summary": "Implemented the selected work unit.",
  "completed": ["Created the CLI command and tests."],
  "notCompleted": [],
  "filesChanged": ["src/cli/commands/example.command.ts"],
  "validationResult": "passed",
  "acceptanceCriteriaResult": "passed",
  "validationCommands": [
    {
      "command": "pnpm validate",
      "result": "passed",
      "summary": "All tests passed."
    }
  ],
  "acceptanceCriteria": [
    {
      "criterion": "Command returns deterministic JSON output.",
      "result": "passed",
      "evidence": "Integration test added."
    }
  ],
  "issues": [],
  "targetStatus": "done"
}
```

9.1 Required top-level fields

Field	Requirement
summary	Required non-empty string. Concise summary of what changed.
validationResult	Required. One of passed, failed, partial, not_run.
acceptanceCriteriaResult	Required. One of passed, failed, partial, not_checked.

9.2 Optional top-level fields

Field	Default / behavior
completed	Defaults to []. Work completed in this execution cycle.
notCompleted	Defaults to []. Work explicitly not completed.
filesChanged	Defaults to []. Relative file paths or areas changed by the agent.
validationCommands	Defaults to []. Captures commands the human/agent claims were run and their result.
acceptanceCriteria	Defaults to []. Optional per-criterion result details.
issues	Defaults to []. Issues or risks found during implementation.
targetStatus	Optional. One of done, needs_review. If omitted, M5 derives the status deterministically.
notes	Defaults to []. Additional compact notes.

9.3 Validation command input

Field	Requirement
command	Required non-empty string.
result	Required. One of passed, failed, partial, not_run.
summary	Optional nullable string.

9.4 Acceptance criterion input

Field	Requirement
criterion	Required non-empty string.
result	Required. One of passed, failed, partial, not_checked.
evidence	Optional nullable string.

9.5 Issue input

Field	Requirement
title	Required non-empty string.
description	Optional nullable string.
severity	Required. One of low, medium, high, critical.
status	Optional. Defaults to open. One of open, resolved.
agentCanFix	Optional boolean. Defaults to true.

10. Completion Decision Rules

The checkpoint result determines the final status of the current work unit. M5 does not mark work done unless the result is explicitly acceptable.

Case	Result
targetStatus omitted and validationResult = passed, acceptanceCriteriaResult = passed, notCompleted is empty, and no open high/critical issue exists	Derive targetStatus = done.
targetStatus omitted and any required completion condition is not satisfied	Derive targetStatus = needs_review.
targetStatus = done and validationResult is not passed	ValidationFailed, exit code 1, no mutation.
targetStatus = done and acceptanceCriteriaResult is not	ValidationFailed, exit code 1, no mutation.

passed	
targetStatus = done and notCompleted has one or more items	ValidationFailed, exit code 1, no mutation.
targetStatus = done and any open high/critical issue exists	ValidationFailed, exit code 1, no mutation.
targetStatus = needs_review	Allowed. The selected work unit becomes needs_review.

Exit code 1 means the checkpoint input is valid JSON and the workflow state is valid, but the submitted completion claim does not pass the completion gate.

11. Dependency Readiness Recalculation

Dependency recalculation happens only after the selected current work unit becomes done. A work unit that becomes needs_review does not release downstream dependencies.

- Blocking dependency types are blocks and requires.
- relates_to dependencies are informational and do not affect readiness.
- A planned work unit becomes ready when every incoming blocks/requires dependency points to a source work unit with status = done.
- M5 may transition only planned -> ready for newly unblocked downstream work units.
- M5 must not transition cancelled, replanned, needs_review, in_progress, or done work units during dependency recalculation.
- Newly ready work units are processed in state.workGraph.workUnits stored order.
- M5 must not create, delete, or reorder dependencies.

12. State Transition Rules

Before	After	Allowed in M5
Work unit: in_progress	Work unit: done	Yes, when checkpoint completion passes.
Work unit: in_progress	Work unit: needs_review	Yes, when checkpoint requires review or completion gate derives needs_review.
Work unit: planned	Work unit: ready	Yes, only for downstream work units unblocked by a done dependency source.
Work unit: ready	Work unit: in_progress	No. M4 aiqt next handles this.
Work unit: needs_review	Work unit: done	No. Future review/checkpoint flow handles this.
Milestone: in_progress	Milestone: done	Yes, when all child work units are done.
Milestone: in_progress	Milestone: ready	Yes, when no child is in_progress/needs_review and at least one child is ready.
Project: in_progress	Project: in_progress	Yes, when more ready or planned work remains.
Project: in_progress	Project: review	Yes, when selected work needs review or all work is done and review/export should run.

After a successful checkpoint, state.currentWorkUnitId is always set to null. If the checkpointed work unit becomes needs_review, state.currentMilestoneId is set to checkpointedWorkUnit.milestoneId. Else, if the checkpointed work unit becomes done and another ready work unit exists after dependency recalculation, state.currentMilestoneId is set to nextReadyWorkUnit.milestoneId. Else, state.currentMilestoneId is set to null. There is no optional or implementation-specific currentMilestoneId behavior in M5.

lastAgentPacket is preserved as the most recent packet metadata. It is not cleared by M5.

13. Canonical Mutation Rules

- aiqt checkpoint mutates state.json and runlog.jsonl only.
- aiqt checkpoint must not mutate project.json.
- One checkpoint record is appended to state.checkpoints per successful checkpoint command.
- The checkpoint record references the currentWorkUnitId and the lastAgentPacket.id.

- The selected work unit status changes from `in_progress` to `done` or `needs_review`.
- Downstream planned work units may become ready after dependency recalculation.
- Milestone statuses are recalculated from child work-unit statuses.
- Project status is recalculated using the M5 state outcome table.
- `state.currentWorkUnitId` is set to null on success.
- `state.lastUpdatedAt` is updated using the command timestamp.
- `state.nextRecommendedCommand` is set using the M5 `nextRecommendedCommand` precedence rule: `needs_review` -> `aiqt review`; `done` with ready work -> `aiqt next`; `done` with no ready work -> `aiqt review`.
- No markdown checkpoint file is created by default.
- `AIQT_SCHEMA_VERSION` remains 0.5.0 unless final implementation review identifies an incompatible canonical schema change.

14. Runlog Rules

A successful checkpoint appends events in this order:

1. `checkpoint.created`
2. `work_unit.status_changed` for the selected current work unit
3. `work_unit.status_changed` for each downstream work unit that becomes ready, in stored work-unit order

```
{
  "id": "EVT-007",
  "type": "checkpoint.created",
  "timestamp": "ISO_DATE",
  "actor": "human",
  "summary": "Checkpoint captured for work unit WU001.",
  "relatedIds": ["PROJECT-001", "M001", "WU001", "C001", "PKT-001"],
  "data": {
    "checkpointId": "C001",
    "workUnitId": "WU001",
    "packetId": "PKT-001",
    "validationResult": "passed",
    "acceptanceCriteriaResult": "passed",
    "targetStatus": "done",
    "nextRecommendedCommand": "aiqt next"
  }
}

{
  "id": "EVT-008",
  "type": "work_unit.status_changed",
  "timestamp": "ISO_DATE",
  "actor": "aiqt",
  "summary": "Work unit WU001 completed.",
  "relatedIds": ["PROJECT-001", "M001", "WU001", "C001"],
  "data": {
    "workUnitId": "WU001",
    "fromStatus": "in_progress",
    "toStatus": "done",
    "reason": "Checkpoint captured."
  }
}
```

- Event IDs must continue from the highest existing EVT id.
- Checkpoint IDs use the C prefix and continue from the highest existing checkpoint ID.
- Failed, blocked, no-input, and example-mode checkpoint attempts must not append runlog events.
- Malformed runlog lines continue to behave according to existing runlog health rules.

15. Command Behavior

15.1 aiqt checkpoint gate order

4. If --example is supplied, handle example mode before reading .aiqt/. Example mode never mutates files.
5. Locate .aiqt/.
6. Read and validate project.json and state.json.
7. Inspect runlog health using existing behavior.
8. If workGraph is empty, evaluate planningContextReady and recommend aiqt update or aiqt plan with exit code 2.
9. Verify currentWorkUnitId exists.
10. Verify currentWorkUnitId references an existing in_progress work unit.
11. Verify lastAgentPacket exists and references the current work unit.
12. If --from-file is missing, return exit code 10 with no mutation.
13. Read and strictly validate CheckpointInputSchema.
14. Derive or validate targetStatus.
15. Apply completion decision rules.
16. Create checkpoint record.
17. Apply status transitions and dependency readiness recalculation.
18. Update current pointers and nextRecommendedCommand.
19. Append runlog events.
20. Return deterministic human or JSON output.

15.1.1 nextRecommendedCommand precedence

M5 applies nextRecommendedCommand in this exact order after successful mutation:

21. If the checkpointed work unit ends as needs_review, nextRecommendedCommand = aiqt review.
22. Else, if the checkpointed work unit ends as done and at least one ready work unit exists after dependency recalculation, nextRecommendedCommand = aiqt next.
23. Else, if the checkpointed work unit ends as done and no ready work unit exists, nextRecommendedCommand = aiqt review.

Review takes priority over continuing when the just-checkpointed unit needs review, even if another unrelated ready work unit exists elsewhere in the graph.

15.2 Human output

Human-readable output should be a concise checkpoint summary, not a markdown report. It should include checkpoint ID, work unit ID, final work unit status, validation result, acceptance criteria result, newly ready work units, changed files, and next recommended command.

15.3 aiqt checkpoint --json

JSON mode returns the existing CommandResult contract and exposes checkpoint details under data.

```
{
  "status": "passed",
  "action": "checkpoint",
  "projectStatus": "in_progress",
  "currentMilestoneId": "M001",
  "currentWorkUnitId": null,
  "summary": "Checkpoint captured for WU001.",
  "completedActions": ["Read project.json", "Read state.json", "Created checkpoint", "Updated workflow state", "Appended runlog events"],
  "changedFiles": [".aiqt/state.json", ".aiqt/runlog.jsonl"],
  "affectedItems": ["PROJECT-001", "M001", "WU001", "C001"],
  "blockingIssues": [],
  "warnings": [],
  "requiresHumanInput": false,
  "nextRecommendedCommand": "aiqt next",
  "data": {
```

```

    "checkpointId": "C001",
    "workUnitId": "WU001",
    "packetId": "PKT-001",
    "fromStatus": "in_progress",
    "toStatus": "done",
    "validationResult": "passed",
    "acceptanceCriteriaResult": "passed",
    "newlyReadyWorkUnitIds": ["WU002"],
    "nextReadyWorkUnitId": "WU002"
  },
  "exitCode": 0
}

```

16. Data and Schema Contracts

16.1 CheckpointInputSchema

```

export const CheckpointInputSchema = z.object({
  summary: z.string().min(1),
  completed: z.array(z.string().min(1)).default([]),
  notCompleted: z.array(z.string().min(1)).default([]),
  filesChanged: z.array(z.string().min(1)).default([]),
  validationResult: z.enum(["passed", "failed", "partial", "not_run"]),
  acceptanceCriteriaResult: z.enum(["passed", "failed", "partial", "not_checked"]),
  validationCommands: z.array(ValidationCommandResultInputSchema).default([]),
  acceptanceCriteria: z.array(AcceptanceCriterionResultInputSchema).default([]),
  issues: z.array(CheckpointIssueInputSchema).default([]),
  targetStatus: z.enum(["done", "needs_review"]).optional(),
  notes: z.array(z.string().min(1)).default([]),
}).strict();

```

16.2 CheckpointSchema

```

export const CheckpointSchema = z.object({
  id: z.string(),
  workUnitId: z.string(),
  packetId: z.string().nullable(),
  summary: z.string(),
  completed: z.array(z.string()),
  notCompleted: z.array(z.string()),
  filesChanged: z.array(z.string()),
  issues: z.array(CheckpointIssueSchema),
  validationResult: z.enum(["passed", "failed", "partial", "not_run"]),
  acceptanceCriteriaResult: z.enum(["passed", "failed", "partial", "not_checked"]),
  validationCommands: z.array(ValidationCommandResultSchema),
  acceptanceCriteria: z.array(AcceptanceCriterionResultSchema),
  finalWorkUnitStatus: z.enum(["done", "needs_review"]),
  nextRecommendation: z.string(),
  createdAt: z.string(),
});

```

CheckpointIssueSchema is a durable domain record stored inside checkpoint records. Issue is the transient CommandResult diagnostic contract used for blocking issues and warnings. They are intentionally different types and must not be merged.

16.3 ID prefixes

Record type	ID prefix	Examples
Checkpoint	C	C001, C002
Runlog Event	EVT-	EVT-007, EVT-008

17. Error Handling and Exit Codes

Case	Exit code	Status	Mutation	Next command
.aiqt/ missing	3	failed	No	aiqt init
project.json or state.json invalid	3	failed	No	null
unsupported future schema version	3	failed	No	null
work graph empty and planningContextReady false	2	blocked	No	aiqt update
work graph empty and planningContextReady true	2	blocked	No	aiqt plan
currentWorkUnitId missing/null	2	blocked	No	aiqt next
currentWorkUnitId references unknown work unit	3	failed	No	aiqt next
current work unit is not in progress	3	failed	No	aiqt next
lastAgentPacket missing or mismatched	3	failed	No	aiqt next
no --from-file and no --example after workflow preconditions pass	10	needs_input	No	aiqt checkpoint --from-file <path>
--example with --json	3	failed	No	null
--from-file path missing/unreadable	3	failed	No	aiqt checkpoint --from-file <path>
--from-file invalid JSON	3	failed	No	aiqt checkpoint --from-file <path>
CheckpointInputSchema validation fails	3	failed	No	aiqt checkpoint --from-file <path>
targetStatus=done but completion gate fails	1	failed	No	aiqt checkpoint --from-file <same path> or aiqt checkpoint
successful checkpoint to done with ready work available	0	passed	Yes	aiqt next
successful checkpoint to done with no ready work available	0	passed	Yes	aiqt review
successful checkpoint to needs_review	0	passed	Yes	aiqt review

Exit code 1 is used when the checkpoint input is valid but the completion claim fails the completion gate. Exit code 2 is reserved for valid workflow-state blocks. Exit code 3 is reserved for invalid input or invalid canonical state. Exit code 10 means a checkpoint input file is required.

For completion-gate failure, the blocking issue must use area = checkpoint, severity = high, agentCanFix = false, and suggestedAction = Correct the checkpoint input and rerun aiqt checkpoint. If input came from --from-file, nextRecommendedCommand must be aiqt checkpoint --from-file <same path>; otherwise it must be aiqt checkpoint.

18. Repository Implementation Scope

Recommended M5 implementation files added or modified:

```
src/
  cli/
    commands/
      checkpoint.command.ts
      register-commands.ts
    options.ts
  schema/
```

```

    checkpoint-input.schema.ts
  services/
    checkpoint-service.ts
  workflow/
    checkpoint-completion-gate.ts
    dependency-readiness.ts
    checkpoint-status-transitions.ts
  state/
    workflow-state-store.ts      # may be extended only if needed
    runlog-store.ts             # extend with M5 event builders
    ids.ts                      # extend for C checkpoint IDs if needed

tests/
  unit/
    checkpoint-input.schema.test.ts
    checkpoint-service.test.ts
    checkpoint-completion-gate.test.ts
    dependency-readiness.test.ts
    checkpoint-status-transitions.test.ts
  integration/
    checkpoint.command.test.ts
    checkpoint-next-flow.test.ts
  fixtures/
    checkpoints/
      valid-done.json
      valid-needs-review.json
      invalid-completion-claim.json
      invalid-schema.json

```

Follow the actual repository layout produced by M1-M4. Do not create duplicate helper locations if equivalent folders already exist.

19. Runtime and Package Impact

- Continue using Node.js, TypeScript, ESM, pnpm, commander, zod, and vitest.
- Do not add AI, GitHub, markdown-parser, graph-database, or source-code execution dependencies.
- Do not add validation-command execution dependencies. M5 records validation results; it does not run them.
- package.json.version may remain unchanged unless implementation intentionally bumps package metadata.
- AIQT_SCHEMA_VERSION should remain 0.5.0 unless final implementation review identifies an incompatible canonical schema change.

20. Testing Scope

- All existing M1, M2, M3, and M4 tests must continue to pass.
- Unit tests must cover CheckpointInputSchema, completion decision rules, dependency readiness recalculation, status transitions, checkpoint record creation, and runlog event builders.
- Integration tests must cover aiqt checkpoint success to done, success to needs_review, --json success, --example, no input, missing active work unit, invalid state, invalid input, completion-gate failure, and no markdown files created.
- Tests must confirm project.json is not mutated by aiqt checkpoint.
- Tests must confirm state.json and runlog.jsonl mutate only on successful checkpoint capture.
- Tests must confirm currentWorkUnitId clears after successful checkpoint.
- Tests must confirm lastAgentPacket is preserved.
- Tests must confirm downstream planned work units become ready only when all blocking dependencies are satisfied.
- Tests must confirm failed or blocked checkpoint attempts leave state.json and runlog.jsonl unchanged.
- Tests must confirm JSON output follows deterministic CommandResult shape.
- Tests must cover the mixed case where the checkpointed work unit becomes needs_review while another unrelated ready work unit exists; expected nextRecommendedCommand is aiqt review.

21. Implementation Tasks

Task	Scope	Acceptance
M5-T1	CLI registration	aiqt checkpoint appears in help; unsupported flags fail deterministically.
M5-T2	CheckpointInputSchema	Valid files pass; unknown keys, invalid enums, and missing required fields fail exit 3.
M5-T3	Checkpoint preconditions	No active currentWorkUnitId blocks with exit 2 and recommends aiqt next. Invalid active state fails exit 3.
M5-T4	Completion gate	targetStatus=done requires passed validation, passed acceptance, no notCompleted, and no open high/critical issues.
M5-T5	Checkpoint service	Creates C IDs, checkpoint records, affected item lists, and deterministic output data.
M5-T6	Status transitions	in_progress -> done/needs_review for selected work unit; currentWorkUnitId clears.
M5-T7	Dependency readiness	done work unit releases downstream planned work units when all blocking dependencies are satisfied.
M5-T8	Runlog events	checkpoint.created and work_unit.status_changed events append in deterministic order.
M5-T9	No mutation failures	Invalid, blocked, no-input, and completion-gate failures leave state and runlog unchanged.
M5-T10	Test suite	Full M1-M5 validation passes with comprehensive unit and integration coverage.

22. Definition of Done

- aiqt checkpoint can be executed after aiqt next creates an in_progress work unit.
- aiqt checkpoint --from-file creates one checkpoint record.
- aiqt checkpoint --json returns deterministic CommandResult JSON.
- aiqt checkpoint --example prints sample JSON and does not mutate files.
- No-input mode returns exit code 10 only after active-work preconditions pass.
- Work unit status changes from in_progress to done or needs_review.
- currentWorkUnitId is cleared after successful checkpoint capture.
- Downstream dependency readiness is recalculated deterministically.
- state.nextRecommendedCommand is set using the M5 nextRecommendedCommand precedence rule: needs_review -> aiqt review; done with ready work -> aiqt next; done with no ready work -> aiqt review.
- project.json is not mutated.
- No markdown files are created by default.
- Runlog events are appended only on successful checkpoint capture.
- Exit codes are consistent with the architecture contract.
- All existing and new tests pass through pnpm validate.
- The project is ready for M6: Review and Export Engines.

23. Agent Execution Packet

```
# AGENT EXECUTION PACKET
```

```
## Role
```

```
You are implementing Milestone 5 of AIQT CLI.
```

```
AIQT is a local TypeScript CLI workflow state engine for projects built with AI coding agents.
```

```

## Starting Point
M4 is complete and merged to main.
Start from: main
Current main commit: c5b42ff
Required existing tag: m4-agent-handoff-packet
Create branch: feat/m5-checkpoint-engine

## Current Work Unit
ID: AIQT-M5
Title: Checkpoint Engine
Objective: Implement aiqt checkpoint so AIQT can capture the result of one agent execution cycle and
advance workflow state.

## Scope
- Implement aiqt checkpoint, aiqt checkpoint --json, aiqt checkpoint --from-file, and aiqt checkpoint --
example.
- Add strict checkpoint input schemas.
- Create checkpoint records in state.json.
- Transition current in_progress work unit to done or needs_review.
- Recalculate dependency readiness after done checkpoints.
- Clear currentWorkUnitId after successful checkpoint.
- Preserve lastAgentPacket metadata.
- Append checkpoint.created and work_unit.status_changed runlog events.
- Return deterministic human and JSON output.
- Preserve M1-M4 behavior.

## Out of Scope
- aiqt review implementation.
- aiqt export implementation.
- Running validation commands automatically.
- Replanning or work graph structure mutation.
- Source-code inspection or modification.
- Markdown file generation.
- External API integration.

## Acceptance Criteria
- pnpm typecheck passes.
- pnpm lint passes.
- pnpm test passes.
- pnpm build passes.
- pnpm validate passes.
- Failed and blocked checkpoint attempts do not mutate state or runlog.
- project.json is not mutated by aiqt checkpoint.

## Required Agent Output
Return: summary, files created, files modified, tests added or changed, validation performed, unresolved
issues, suggested next step.

```

Appendix A - Minimum Test Matrix

Scenario	Expected result
aiqt checkpoint --example outside .aiqt/	Print sample JSON, exit 0, no mutation.
aiqt checkpoint --example --json	Exit 3, no mutation.
.aiqt/ missing	Exit 3, recommend aiqt init.
invalid project.json/state.json	Exit 3, no mutation.
empty workGraph and planningContextReady=false	Exit 2, recommend aiqt update.
empty workGraph and planningContextReady=true	Exit 2, recommend aiqt plan.
workGraph exists but currentWorkUnitId=null	Exit 2, recommend aiqt next.
currentWorkUnitId unknown	Exit 3, no mutation.
current work unit not in progress	Exit 3, no mutation.
lastAgentPacket missing or mismatched	Exit 3, no mutation.
active work unit but no --from-file	Exit 10, no mutation.
invalid checkpoint JSON	Exit 3, no mutation.
unknown nested key in checkpoint input	Exit 3, no mutation.

targetStatus=done with validationResult=failed	Exit 1, no mutation.
targetStatus=done with notCompleted entries	Exit 1, no mutation.
targetStatus=done with open critical issue	Exit 1, no mutation.
valid done checkpoint	Append checkpoint, selected work unit done, currentWorkUnitId null.
valid needs_review checkpoint	Append checkpoint, selected work unit needs_review, recommend aiqt review.
needs_review checkpoint while another ready work unit exists	Append checkpoint, keep/recommend aiqt review because review takes priority.
done checkpoint unlocks downstream work	Downstream planned work unit becomes ready.
done checkpoint with next ready work	Recommend aiqt next.
all work done	Project moves to review and recommends aiqt review.
project.json mutation check	project.json unchanged after checkpoint.
no markdown files check	No AIQT.md, AGENTS.md, CLAUDE.md, docs/, .milestones/, or .tasks/ created.
runlog event order	checkpoint.created before work_unit.status_changed.
full flow init -> update -> plan -> next -> checkpoint	Workflow advances from in_progress to next action.

Appendix B - Example Checkpoint Input File

```
{
  "summary": "Implemented the bounded work unit from the M4 packet.",
  "completed": [
    "Added the command handler.",
    "Added unit and integration tests."
  ],
  "notCompleted": [],
  "filesChanged": [
    "src/cli/commands/example.command.ts",
    "tests/integration/example.command.test.ts"
  ],
  "validationResult": "passed",
  "acceptanceCriteriaResult": "passed",
  "validationCommands": [
    {
      "command": "pnpm validate",
      "result": "passed",
      "summary": "Typecheck, lint, and tests passed."
    }
  ],
  "acceptanceCriteria": [
    {
      "criterion": "The command returns deterministic JSON output.",
      "result": "passed",
      "evidence": "Covered by integration tests."
    }
  ],
  "issues": [],
  "targetStatus": "done",
  "notes": ["No unresolved implementation issues."]
}
```

Appendix C - Example JSON Output

```
{
  "status": "passed",
  "action": "checkpoint",
  "projectStatus": "in_progress",
  "currentMilestoneId": "M001",
  "currentWorkUnitId": null,
  "summary": "Checkpoint captured for WU001.",
  "completedActions": [
    "Read project.json",
```

```
    "Read state.json",
    "Created checkpoint",
    "Updated workflow state",
    "Appended runlog events"
  ],
  "changedFiles": [".aiqt/state.json", ".aiqt/runlog.jsonl"],
  "affectedItems": ["PROJECT-001", "M001", "WU001", "C001"],
  "blockingIssues": [],
  "warnings": [],
  "requiresHumanInput": false,
  "nextRecommendedCommand": "aiqt next",
  "data": {
    "checkpointId": "C001",
    "workUnitId": "WU001",
    "packetId": "PKT-001",
    "fromStatus": "in_progress",
    "toStatus": "done",
    "validationResult": "passed",
    "acceptanceCriteriaResult": "passed",
    "newlyReadyWorkUnitIds": ["WU002"],
    "nextReadyWorkUnitId": "WU002"
  },
  "exitCode": 0
}
```